

JavaScript Class Tasks — Complete Solutions

Playlist, FlipkartProduct, and Movie class exercises (Tasks 1-13)

Task 1

Declare a JavaScript class named Playlist with a constructor accepting name and songs (an array). Create an instance for a Spotify playlist called 'Chill Vibes' with 3 song names.

```
class Playlist {
  constructor(name, songs) {
    this.name = name;
    this.songs = songs;
  }
}

const chillVibes = new Playlist("Chill Vibes", [
  "Blinding Lights",
  "Perfect",
  "Until I Found You"
]);

console.log(chillVibes);
```

Task 2

Same as Task 1: create a Playlist class and a 'Chill Vibes' instance containing 3 songs.

```
class Playlist {
  constructor(name, songs) {
    this.name = name;
    this.songs = songs;
  }
}

const chillVibes = new Playlist("Chill Vibes", [
  "Blinding Lights",
  "Perfect",
  "Until I Found You"
]);

console.log(chillVibes);
```

Task 3

Declare Playlist with name and songs, create an instance for a favorite Spotify playlist, and log it to the console.

```
class Playlist {
  constructor(name, songs) {
    this.name = name;
    this.songs = songs;
  }
}

const myPlaylist = new Playlist("My Favorites", [
  "Shape of You",
  "Someone Like You",
  "As It Was"
]);

console.log(myPlaylist);
```

Task 4

Add `displayPlaylist()` to `Playlist` so it logs the playlist name and all song titles in a formatted string.

```
class Playlist {
  constructor(name, songs) {
    this.name = name;
    this.songs = songs;
  }

  displayPlaylist() {
    console.log(`Playlist: ${this.name}`);
    console.log(`Songs: ${this.songs.join(", ")}`);
  }
}

const chillVibes = new Playlist("Chill Vibes", [
  "Blinding Lights",
  "Perfect",
  "Until I Found You"
]);

chillVibes.displayPlaylist();
```

Task 5

Add `displayPlaylist()` to `Playlist` so it logs the playlist name and all song titles in a formatted string.

```
class Playlist {
  constructor(name, songs) {
    this.name = name;
    this.songs = songs;
  }

  displayPlaylist() {
    console.log(`Playlist: ${this.name} | Songs: ${this.songs.join(", ")}`);
  }
}

const chillVibes = new Playlist("Chill Vibes", [
  "Blinding Lights",
  "Perfect",
  "Until I Found You"
]);

chillVibes.displayPlaylist();
```

Task 6

Add `getSongCount()` to `Playlist`, return the total number of songs, call it, and display the result.

```
class Playlist {
  constructor(name, songs) {
    this.name = name;
    this.songs = songs;
  }

  getSongCount() {
    return this.songs.length;
  }
}

const chillVibes = new Playlist("Chill Vibes", [
  "Blinding Lights",
  "Perfect",
  "Until I Found You"
]);
```

```
console.log("Song count:", chillVibes.getSongCount());
```

Task 7

Create FlipkartProduct with title, price, and rating. Add getDiscountedPrice(discountPercent) using the given formula.

```
class FlipkartProduct {
  constructor(title, price, rating) {
    this.title = title;
    this.price = price;
    this.rating = rating;
  }

  getDiscountedPrice(discountPercent) {
    return this.price - (this.price * discountPercent / 100);
  }
}

const product = new FlipkartProduct("Redmi Note 12 Pro", 18999, 4.5);

console.log(product.getDiscountedPrice(10)); // 17099.1
```

Task 8

Same as Task 7: create FlipkartProduct with getDiscountedPrice(discountPercent).

```
class FlipkartProduct {
  constructor(title, price, rating) {
    this.title = title;
    this.price = price;
    this.rating = rating;
  }

  getDiscountedPrice(discountPercent) {
    return this.price - (this.price * discountPercent / 100);
  }
}

const product = new FlipkartProduct("Redmi Note 12 Pro", 18999, 4.5);

console.log(product.getDiscountedPrice(20)); // 15199.2
```

Task 9

Create FlipkartProduct and add displayDetails() that prints details like 'Redmi Note 12 Pro - ₹18999 - 4.5★'.

```
class FlipkartProduct {
  constructor(title, price, rating) {
    this.title = title;
    this.price = price;
    this.rating = rating;
  }

  displayDetails() {
    console.log(`${this.title} - ₹${this.price} - ${this.rating}★`);
  }
}

const product = new FlipkartProduct("Redmi Note 12 Pro", 18999, 4.5);

product.displayDetails();
```

Task 10

Refactor the Movie function constructor and prototype method into a JavaScript class.

```
class Movie {
  constructor(title, genre) {
    this.title = title;
    this.genre = genre;
  }

  display() {
    console.log(this.title + " - " + this.genre);
  }
}

const movie = new Movie("Inception", "Sci-Fi");
movie.display();
```

Task 11

Same as Task 10: refactor Movie into a class while keeping the same behavior.

```
class Movie {
  constructor(title, genre) {
    this.title = title;
    this.genre = genre;
  }

  display() {
    console.log(this.title + " - " + this.genre);
  }
}

const movie = new Movie("Inception", "Sci-Fi");
movie.display();
```

Task 12

Extend Playlist with addSong(songName), demonstrate by adding two songs, then display the updated playlist.

```
class Playlist {
  constructor(name, songs) {
    this.name = name;
    this.songs = songs;
  }

  addSong(songName) {
    this.songs.push(songName);
  }

  displayPlaylist() {
    console.log(`Playlist: ${this.name}`);
    console.log(`Songs: ${this.songs.join(", ")}`);
  }
}

const chillVibes = new Playlist("Chill Vibes", [
  "Blinding Lights",
  "Perfect",
  "Until I Found You"
]);

chillVibes.addSong("Heat Waves");
chillVibes.addSong("Levitating");

chillVibes.displayPlaylist();
```

Task 13

Refactor FlipkartProduct by renaming displayDetails() to showProductInfo() and update the call.

```
class FlipkartProduct {
  constructor(title, price, rating) {
    this.title = title;
    this.price = price;
    this.rating = rating;
  }

  showProductInfo() {
    console.log(` ${this.title} - ₹${this.price} - ${this.rating} ⭐`);
  }
}

const product = new FlipkartProduct("Redmi Note 12 Pro", 18999, 4.5);

product.showProductInfo();
```